CLI Options in the Wild

Compter l'évolution des options en ligne de commande

Théophile BERTHELOT Loïs CHOW-CHINE-GALINAUD Yoan FRANÇOIS Baptiste GUERIF

Projet PROJ104 – Encadrante : Xhevahire TËRNAVA

2025/2026

Objectif du projet

Comprendre comment les interfaces en ligne de commande évoluent dans le temps.

L'idée centrale :

- récupérer l'aide de programmes CLI ;
- détecter et compter leurs options ;
- comparer les résultats entre programmes et entre versions ;
- visualiser l'évolution de cette complexité.

Exemples : `ls`, `cat`, `grep`, `sed`, `sort`, `curl`, `git`, `coreutils`.

Pourquoi les options CLI ?

Les options sont une mesure concrète de la richesse fonctionnelle d'un outil.

- Elles montrent l'ajout progressif de fonctionnalités.
- Elles révèlent la complexification des interfaces.
- Elles permettent de comparer outils Unix historiques et outils modernes.
- Elles exposent les limites de la documentation analysable automatiquement.

Question de fond

Est-ce que les outils en ligne de commande deviennent plus complexes au fil du temps ?

Fonctionnement général

Pipeline du projet :

1. choisir un programme ou un dépôt Git ;
2. exécuter la commande d'aide adaptée ;
3. extraire les options avec un parseur Python ;
4. compter les options uniques ;
5. exporter les résultats en CSV ou JSON ;
6. produire des graphiques et visualisations.

Le projet contient aussi une interface graphique pour tester rapidement le compteur.

Ce que le compteur analyse

Le parseur repère principalement :

- les options courtes : `-h`, `-v`, `-o` ;
- les options longues : `-help`, `-version`, `-output` ;
- les variantes d'aide selon les programmes.

Modes de sortie :

- affichage terminal ;
- JSON ;
- CSV ;
- intégration possible avec des graphiques.

Limite : le projet extrait les options visibles dans l'aide, sans analyser toute la grammaire CLI.

Difficulté : les aides ne sont pas uniformes

Tous les programmes ne documentent pas leurs options de la même manière.

- `-help` suffit pour certains programmes.
- D'autres nécessitent `-h`, `-help`, `-usage`, `-help all`, `-help full`.
- Certaines options sont groupées.
- Certaines aides contiennent des sous-commandes.
- Les formats changent parfois entre versions.

Cela oblige à adapter la commande d'aide et les règles de parsing.

Difficulté : voyager dans le temps

Pour mesurer l'évolution d'un programme, il faut reconstruire d'anciennes versions.

1. cloner un dépôt ;
2. sélectionner les tags pertinents ;
3. choisir une version par année ;
4. compiler chaque version ;
5. exécuter le compteur ;
6. stocker les résultats.

Problème principal

Les commandes de build et les dépendances changent selon les versions.

Enjeux techniques

Les principaux enjeux techniques sont :

- parsing robuste de textes d'aide hétérogènes ;
- prise en charge des alias et variantes d'aide ;
- gestion des sous-commandes ;
- automatisation Git sur plusieurs tags ;
- compilation de versions anciennes ;
- export propre des données ;
- visualisation lisible des résultats.

Le projet combine parsing, automatisation système et analyse de données.

Répartition des tâches

Membre	Rôle principal
Yoan FRANÇOIS	Interface graphique et intégration des graphes
Baptiste GUERIF	Code du compteur, parsing, restructuration du dépôt
Loïs CHOW-CHINE-GALINAUD	Analyse des dépôts Git et automatisation temporelle
Théophile BERTHELOT	Analyse de données, sources, graphiques temporels

Travail commun :

- choix des règles de comptage ;
- tests sur programmes réels ;
- documentation ;
- préparation des livrables.

Avancement actuel

Ce qui est déjà présent dans le dépôt :

- compteur Python fonctionnel ;
- options de sortie JSON et CSV ;
- documentation utilisateur ;
- règles de comptage ;
- interface graphique Tkinter ;
- premiers graphiques ;
- script de téléchargement et test de versions Git ;
- planning et suivi de projet.

Structure principale : `src/`, `docs/`, `Visualisation/`, `presentations/`.

Planning de fin de projet

Fin mai – début juin

Corriger les cas d'erreur, documenter les limites, stabiliser le pipeline Git, sélectionner les programmes à analyser.

Mi-juin

Lancer les analyses finales, générer les CSV/JSON, produire les graphiques Matplotlib, comparer l'évolution du nombre d'options.

Dernière semaine

Finaliser README et documentation, préparer poster et présentation, répéter la soutenance.

Ce qu'il reste à faire

Priorités techniques

- améliorer le support des programmes complexes ;
- gérer `-help full` et `-help all` ;
- fiabiliser les builds historiques ;
- ajouter des recettes par année ou tag ;
- sauvegarder les résultats.

Priorités analyse

- choisir les programmes finaux ;
- vérifier manuellement un échantillon ;
- produire les graphes finaux ;
- interpréter les tendances observées.

Enjeux sociétaux

Les outils CLI restent centraux pour :

- l'administration système ;
- le développement logiciel ;
- la cybersécurité ;
- l'automatisation industrielle ;
- la recherche reproductible.

Comprendre leur complexité aide à évaluer l'accessibilité, la charge cognitive, la maintenabilité et la qualité de la documentation technique.

Enjeux environnementaux

Points négatifs possibles :

- compiler de nombreuses versions consomme du temps CPU ;
- télécharger des dépôts et dépendances multiplie les transferts réseau ;
- reconstruire des environnements anciens peut être coûteux.

Pistes de réduction :

- limiter le nombre de programmes analysés ;
- choisir une version par année plutôt que tous les tags ;
- mettre en cache les dépôts et résultats ;
- éviter de relancer les builds inutiles ;
- documenter les échecs non reproductibles.

Apport du projet

Le projet produit :

- un outil de comptage réutilisable ;
- une méthode pour comparer des outils CLI ;
- des données exportables ;
- des visualisations ;
- une réflexion sur l'évolution de la complexité logicielle.

Il relie histoire des logiciels, ergonomie des interfaces, automatisation et analyse de données.

Conclusion

CLI Options in the Wild cherche à mesurer une évolution souvent invisible :

La croissance de la complexité des interfaces en ligne de commande.

Le projet avance sur trois axes :

- compter correctement les options ;
- automatiser l'analyse dans le temps ;
- rendre les résultats compréhensibles par des graphes et une présentation claire.

Objectif final : une analyse fiable, documentée et présentable de plusieurs programmes CLI réels.